

Heterogeneous Edge devices

Abstract—

- 1) There are m edge devices, n users and a single controller.
- 2) The users can reach both the controller and the edge devices via a single wireless network. However, the controller can reach the edge devices via a wired network.
- 3) The controller is responsible for routing job requests at each time step. We denote the number of requests by k , note that the value of k changes with each time step.
- 4) The controller keeps track of the amount of time taken by each edge device using its own model of computation. The edge devices send a message to the controller once a job is completed.
- 5) The state used by the controller consists of static and dynamic configurations. The static state used consists of (CPU speed, memory, GPU speed, GPU memory, number of cores), and the dynamic state consists of (load, CPU utilization, GPU utilization, memory utilization, core utilization, GPU memory utilization).
- 6) Since the edge's state has different dimensions, we use an edge encoder to convert it into same dimension.
- 7) The output of the edge encoder along with application-level parameters (flops, file size and signal strength) are given to the reinforcement learning system. Thus, the input of the RL has the dimensions $(ml + 3k)$.
- 8) The output of the RL is nk , but the last layer used is sigmoid, as multiple edge devices can be chosen. In case, a service is not selected by the sigmoid, then an argmax would be taken for the output of that service to choose at least one edge.
- 9) The objective is to complete as many services within the deadline as possible, with a higher priority assigned to the larger jobs in proportion to their job lengths.
- 10) The reward consists of the su

I. INTRODUCTION

In recent years, the proliferation of edge computing has revolutionized the way latency-sensitive applications are deployed and managed. Edge computing brings computational resources closer to end users, significantly reducing latency and enabling real-time processing for applications such as augmented reality (AR), autonomous vehicles, healthcare monitoring, and industrial automation. Unlike traditional cloud computing architectures, edge computing minimizes the dependency on centralized data centers by leveraging distributed edge nodes. This shift is crucial for latency-sensitive applications, as even minor delays in data processing can lead to degraded performance or critical failures in scenarios requiring instantaneous decision-making.

However, the efficient scheduling of workloads on edge servers remains a critical challenge in edge computing environments. With the increasing number of latency-sensitive applications, edge servers often face uneven workload distributions, leading to performance bottlenecks and increased response times. To address this, a controller-based approach is proposed to optimize workload scheduling. The controller acts as an intermediary, receiving user queries, analyzing the current workload on available edge servers, and forwarding tasks to the server with the least workload. This dynamic allocation ensures a balanced distribution of workloads, reduces processing delays, and enhances the overall efficiency of the system, making it well-suited for latency-critical scenarios.

Several existing studies have explored workload scheduling and optimization in edge computing environments. Techniques such as load balancing, task offloading, and resource provisioning have been widely proposed to mitigate latency and improve system efficiency. For instance, [?] investigated distributed scheduling mechanisms to balance workloads across edge servers, while [?] introduced machine learning-based models for predicting server loads and optimizing task assignments. Despite these advancements, many existing approaches struggle with real-time adaptability to dynamic workloads and the overhead associated with centralized management. Moreover, few studies emphasize integrating a centralized controller for intelligent workload redistribution across edge servers, leaving a gap in achieving both scalability and low latency in diverse application scenarios.

To address these challenges, we propose a controller-based workload scheduling mechanism that leverages queueing theory to model and optimize task handling at the controller. Specifically, the controller queue is modeled as an $M/M/1$ queue with arrival rate λ . By employing this probabilistic model, we can analytically determine key performance metrics such as the average waiting time, queue length, and system utilization. These metrics provide insights into optimizing the controller's task forwarding strategy, ensuring minimal delays even under high workload scenarios.

Our approach dynamically adjusts task assignments based on real-time queue states and server workload distributions, achieving a scalable and low-latency solution for edge computing environments. By integrating queueing theory into workload scheduling, we not only enhance the adaptability of the controller to fluctuating workloads but also reduce the risk of bottlenecks, making the system resilient to varying application demands.

In this work, we address the critical challenges of workload scheduling and latency optimization in edge computing environments by proposing a novel controller-based scheduling mechanism. The key contributions of our work are summarized as follows:

- 1) **Controller-Based Workload Scheduling:** We propose a

centralized controller framework that dynamically assigns tasks to edge servers based on real-time workload and queue state, ensuring efficient utilization of computational resources and minimizing processing delays.

- 2) Queue Modeling for Task Management: The controller queue is modeled as an M/M/1 queue, enabling the analytical evaluation of performance metrics such as average waiting time, queue length, and system utilization. This modeling provides a robust foundation for optimizing task scheduling strategies under varying A workload conditions.
- 3) Dynamic Workload Adaptation: Our approach dynamically adjusts task distribution to accommodate fluctuating workloads, ensuring scalability and resilience in highly dynamic edge computing environments.
- 4) Comprehensive Evaluation: We perform extensive simulations to validate the effectiveness of the proposed approach, comparing it against existing scheduling mechanisms. Results demonstrate significant improvements in reducing response times, balancing workloads, and achieving high system throughput.

By addressing both theoretical and practical aspects of workload scheduling in edge computing, this work provides a scalable and adaptable solution for latency-critical applications.

II. PROBLEM STATEMENT

In this section, we formulate our problem. Our framework incorporates the following components:

- The dynamic state of each edge server $e_i \in \mathcal{E}$ at any timestep t is identified by an extended tuple:

$$(\lambda_i^{u(t)}, \lambda_i^{d(t)}, \mathcal{M}_i^t, \mathcal{U}_i^t, \ell_i^t, \mathcal{R}_i^t, \mathcal{L}_i^t, \mathcal{C}_i^t, \mathcal{F}_i^t, \mathcal{U}_{\text{CPU},i}^t, \mathcal{F}_{\text{GPU},i}^t, \mathcal{U}_{\text{GPU},i}^t, \mathcal{M}_{\text{GPU},i}^t, \mathcal{L}_{\text{max},i}^t),$$

where:

- CPU clock speed.
- Total RAM
- GPU clock speed.
- GPU Memory
- Number of cores
- For each core, CPU percentage.
- Memory utilization
- GPU usage
- GPU memory utilization
- Number of user accessing the machine. (Application specific, Process-1, Process-2, Process-3)
- Total Bandwidth
- $\lambda_i^{u(t)}, \lambda_i^{d(t)}$ denote the uplink and downlink bandwidths at timestep t .
- $\mathcal{M}_i^t$ and $\mathcal{U}_i^t$ denote the memory and CPU utilization at timestep t .
- ℓ_i^t is the number of active users accessing e_i at timestep t .
- $\mathcal{R}_i^t$ is the total RAM available on the edge server.
- $\mathcal{L}_i^t$ is the current load on the edge server.
- $\mathcal{C}_i^t$ represents the number of cores used.
- $\mathcal{F}_i^t$ denotes the frequency of each core.

- $\mathcal{U}_{\text{CPU},i}^t$ is the background CPU utilization calculated using an inbuilt function.
- $\mathcal{F}_{\text{GPU},i}^t$ is the frequency of the GPU.
- $\mathcal{U}_{\text{GPU},i}^t$ is the utilization of the GPU in the background calculated using an inbuilt function.
- $\mathcal{M}_{\text{GPU},i}^t$ is the GPU memory available on the edge server.
- $\mathcal{L}_{\text{max},i}^t$ is the maximum load capacity of the edge server.

A set of users $U = \{u_1, u_2, \dots, u_n\}$, each at specific locations $L = \{\ell_1, \ell_2, \dots, \ell_n\}$, require edge servers for the execution of their respective services $S = \{s_1, s_2, \dots, s_n\}$, where s_j may vary for each user u_i at location ℓ_i .

User Device and State: Each user device is characterized by a tuple: $(L, \Lambda^u, \Lambda^d, \Theta_u)$, where:

- L is the user's location.
- Λ^u and Λ^d denote the uplink and downlink bandwidths, respectively.
- Θ_u represents the queuing delays experienced during the transfer of data to the controller and edge server. This includes waiting times at both the controller and edge server, modeled using M/M/1 queues with arrival rate λ and service rate μ .

Each service s_j is characterized by a 4-tuple: $(\mathcal{I}_j, \mathcal{O}_j, \Gamma_j, \Theta_j)$, where:

- $\mathcal{I}_j$ represents a set of input parameters required by the service, which are transferred from the user device to the controller and subsequently to the edge server for execution.
- $\mathcal{O}_j$ denotes a set of output parameters generated by the service after execution, which are transferred back from the edge server to the controller and then to the user device.
- Γ_j includes the characteristics of the input parameters that influence the computation time of s_j . These characteristics can include size, complexity, or other dynamic factors and may vary across different services.
- Θ_j represents the waiting time parameters associated with the queuing delays during the transfer and execution of the service. This includes waiting times at both the controller and edge server, modeled using M/M/1 queues with arrival rate λ and service rate μ .

Transfer of service-input: A set of input parameters $\mathcal{I}_j$ for the service s_j must be transferred from the user device to a central controller and then from the controller to the edge server for execution. This transfer time includes transmission, propagation latency, and waiting time due to queuing at the controller or edge server. The transfer process involves four key events:

- 1) Uploading the input file from the user device to the controller.
- 2) Propagating the file from the user device to the controller (represented by a function of the user location $\rho_i(L)$).
- 3) Waiting at the controller queue modeled as an M/M/1 queue with arrival rate λ and service rate μ .
- 4) Forwarding the input file from the controller to the edge server.

Execution of service: Once an edge server receives the input of a service, the service is executed on it. The computation time $\mathcal{C}(e_i, s_j)$ at e_i is a function of various parameters of s_j , the dynamic state of the edge server e_i , and the queuing time at the edge server (also modeled as an $M/M/1$ queue with arrival rate λ and service rate μ).

Transfer of service output: The output parameters $\mathcal{O}_j$ of the service s_j are transferred back from the edge server to the central controller and then to the user device. This process is similar to the transfer of the service input and includes waiting time at the edge server and controller queues.

Assumptions

- 1) There is a controller that decides which workload is sent to which edge server.
- 2) There are two distinct deadlines, L_1 and L_2 :
 - If the response is received before L_1 , no penalty is applied.
 - If the response is received after L_2 , the highest penalty is applied.
- 3) Each edge device consists of both a CPU and a GPU. The actual execution time depends on the current workload of the edge device and the type of service being executed.

III. METHODOLOGY

We use reinforcement learning strategies to satisfy our two main objectives

- 1) Satisfy maximum number of service requests
- 2) Minimise the service latency

We define step and episodic reward to achieve these objectives.

A. Step

Step is a set of events in which the RL algorithm runs and allocates the service requests to edge servers.

The controller receives a batch of service request and each service is allocated to multiple edge servers. This redundant allocation aids in executing the service request in minimum time possible. Lets say a batch has 's' service requests and 'e' servers are selected by RL for execution.

B. Episode

Execution of k steps makes up an episode. An episode is said to be completed only when the last task of kth step has been executed.

C. Assumptions

- 1) The inference time of RL algorithm is much smaller than the execution time of the service.
- 2) After every step, virtual state of Edge Server is updated and is known to the controller as it is continuously sensing the status of each edge server in the network.

- 3) The service characteristics are also known to edge server through the controller and therefore waiting time can be estimated even though the service request is yet to arrive to the edge server via user.
- 4) The arrival of service requests is random and therefore it is safe to say that the arrival rate follows Poisson's distribution.
- 5) The remaining service time of a request is independent of the service time that has already been passed. For this memoryless property and sake of simplicity in mathematical calculations we take service rate to follow an exponential distribution.

D. Waiting Time

Waiting time can be defined as the time a service has to spend in the queue of edge server before it can be executed. The waiting time can be estimated by using queuing theory.

We use M/M/1 queue model to estimate the waiting time of each edge server that has been selected by the RL algorithm in a step. We take the minimum value among the calculated waiting times of edge server as our waiting time.

$$W_{edge} = \frac{\lambda}{\mu(\mu - \lambda)}$$

where,

λ is the arrival rate

μ is the service rate

$$W_{step} = \min(W_i) \quad \forall e_i \in \mathcal{E}$$

where,

W_i is the waiting time of i^{th} edge server.

E. Reward Function

We define two rewards namely Step Reward and Episodic Reward.

1) *Step Reward:* After completion of every step the RL agent receives a step reward.

$$R_{step} = \sum_{i=1}^d (l_i - \frac{l_{total}}{m}) - W_{step}$$

where,

R_{step} is the step reward

d is the no of redundant edge servers in a step

m is the no of total edge servers

l_i is queue length of the i^{th} edge server

l_{total} is the summation of queue length of all the edge servers in the system

W_{step} is the Waiting time in a step.

The proposed step reward dynamically captures the load of edge server and pushes the algorithm to balance the load on all edge servers. It also takes into account the waiting latency

and tries to minimize it.

However, in the long term the load balancing factor would force the algorithm to have the same number of service requests allocated to each edge server which might not be the optimal solution as both the service requests and edge servers are heterogeneous and it may be possible that the same service takes different execution times in different edge servers.

2) *Episodic Reward*: After all the tasks of the k^{th} step are completed, the agent would receive an episodic reward.

$$R_{episode} = \sum_{i=0}^k (\gamma^i R_{step}^{i+1}) + \omega - W_{avg}$$

where

$R_{episode}$ is episodic reward

γ is discount factor

k is number of steps in an episode

R_{step} is step reward

ω is degree of satisfaction

W_{avg} is the average waiting time.

a) *Degree of Satisfaction (Definition)*: Degree of Satisfaction can be defined as the ratio of number of satisfied requests in an episode to total number of requests recieved in an episode.

$$\omega = \frac{\text{Number of Satisfied Requests}}{\text{Total Number of Requests}}$$

The value of degree of satisfaction lies between 0 and 1 with 0 being the case when no request is satisfied and 1 when all the requests are satisfied within the deadline.

If a request is satisfied in slightly more time than the deadline (Ψ) then we get a value between 0 and 1. We define this time difference as Deadline threshold (κ).

Let total latency be equal to $\mathbb{L}$ and $\mathcal{B}$ denote if the request is satisfied.

$$\omega = \begin{cases} = 0 & \text{if } \mathcal{B}_{all} = 0 \\ 0 \leq \omega \leq 1 & \text{if } \mathbb{L} + \kappa \leq \Psi \\ = 1 & \mathcal{B}_{all} = 1 \ \& \ \mathbb{L} \leq \Psi \end{cases}$$

IV. EXPERIMENTAL SETUP

A. Modeling Computational Latency:

To accurately replicate computational delay, we conducted rigorous testing on two platforms:

System 1: Intel Xeon 6226R processor with 32 cores at a base clock of 1900 MHz, 256 GB RAM, and an NVIDIA RTX A5000 GPU (24 GB GDDR6 memory).

System 2: AMD Ryzen 9 7950X processor with 16 cores at a base clock speed of 400 MHz, and an NVIDIA GeForce RTX 4080 graphics card (16 GB GDDR6X memory).

B. Workload Description

We select three various applications that are typical compute-intensive tasks (object detection, instance segmentation and audio processing). We executed various numbers of concurrent runs on each of the applications in order to simulate the actual deployment scenarios. We specifically run the above application with all different combinations with varying number of instances of each application and note down the execution time of per-image/per-audio data as well as the execution time of the specific application when running with other applications in parallel. This helps us in modeling heterogeneity in terms of system, applications and compute-intensive tasks. In addition, these services cover two types of input modalities: images and audio. We tested our approach using benchmark datasets: the COCO dataset for visual tasks and the LibriSpeech dataset for audio tasks, which is widely used for speech recognition evaluations.

This method allowed us to rigorously analyze the effect of simultaneous execution and resource competition to compute latency.

C. Data Collection Methodology

For each run of the experiment, we collected traces of fine-grained execution, logging fine-grained and coarse-grained performance measures:

Fine-grained Metrics:

- Per-data-item processing latency (e.g., per-image inference latency for image detection applications, or per-audio-segment processing latency for audio applications).
- Data-specific features that impact the execution time, such as image size (resolution and file size) and audio duration.

Coarse-grained Metrics:

- Total program running time for all combinations of parallel instances.
- System-level resource consumption metrics such as CPU utilization (%), GPU utilization (%), memory usage (RAM and GPU VRAM, GB), number of active CPU cores, and GPU memory usage were recorded during each experimental run.

These were measured systematically so as to be the same in all experimental conditions.

All scripts and detailed data collection procedures are available in our public GitHub repository.

<https://github.com/amardeep786/Regressors>

D. Computation latency prediction

Following the gathering of thorough performance information from extensive parallel execution testing, we built latency estimation models with Multi-Layer Perceptron (MLP) regressors. In particular, we ran three compute-bound applications—object detection, instance segmentation, and speech-to-text—within numerous parallel configurations (single, pairwise, and triple concurrent runs), systematically changing the number of concurrently executed instances between one and five per application. For every execution scenario, we logged

detailed inference times independently for every application: image-based applications (object detection and instance segmentation) had inference latencies generally between 5 to 20 ms per image, and speech-to-text processing had latencies between 60 to 300 ms per audio sample. A single regression model would struggle to fit different latency distributions so well because error minimization would overemphasize the greater absolute errors of audio processing at the expense of the precision of vision tasks for which smaller errors are most important. To simulate the impact of concurrent processes on each application, we incorporated system state attributes of CPU and GPU utilization and RAM utilization into the input of each application-specific model. These attributes are indicators of resource contention, reflecting the impact of concurrent workloads on the executing process. For instance, RAM utilization serves as an indicator of memory contention caused by other applications, effectively capturing the impact of parallel processes on the processing time.

So we trained independent MLP regression models for each type of application, the input parameters consisted of the number of running application instances, application-specific parameters (e.g., image sizes or audio lengths), hardware details (CPU cores, base frequency, RAM size, GPU memory), and real-time system usage metrics (CPU/GPU usage percent). These models helped accurately measure how resource competition impacts delays across different workloads. They provide a clear understanding of computer performance in various hardware setups and multitasking environments, ensuring better performance analysis and optimization.

E. Modeling Waiting Time

Waiting time is a critical metric in process scheduling and performance analysis. It represents the total duration a process spends in the ready queue, waiting to be assigned CPU resources, during its entire execution lifecycle. Accurate modeling of waiting time allows system designers and researchers to evaluate the efficiency of scheduling algorithms and identify potential bottlenecks in system resource allocation.

In modern Linux systems, the process waiting time can be determined by leveraging the `proc` filesystem, a virtual filesystem that provides detailed runtime information about processes and system status. By parsing specific files within `/proc`, such as `/proc/uptime` and `/proc/[pid]/stat`, it is possible to extract the necessary parameters to compute the waiting time of any process.

PROCEDURE FOR CALCULATING WAITING TIME

The procedure for calculating the waiting time of a process using the `proc` filesystem is as follows:

- 1) **Obtain System Uptime:** Read the first value from `/proc/uptime`, which provides the system uptime in seconds since the last boot. To convert this value to clock ticks, multiply it by the number of clock ticks per second (HZ), which can be obtained programmatically using `sysconf(_SC_CLK_TCK)`.

$$U = \text{uptime (seconds)} \times \text{HZ}$$

- 2) **Determine Process Start Time:** Extract the `starttime` field from `/proc/[pid]/stat`, which indicates the time (in clock ticks) after system boot when the process was started.
- 3) **Calculate Elapsed Time Since Process Start:** The elapsed time E (in clock ticks) since the process began is computed as:

$$E = U - \text{starttime}$$

where U is the current uptime in clock ticks and `starttime` is the process start time in clock ticks.

- 4) **Retrieve CPU and I/O Times:** From `/proc/[pid]/stat`, obtain the following fields (all in clock ticks):
 - U_t : User mode CPU time consumed by the process.
 - K_t : Kernel mode CPU time consumed by the process.
 - CU_t : User mode CPU time consumed by terminated child processes.
 - CK_t : Kernel mode CPU time consumed by terminated child processes.

Additionally, from `/proc/[pid]/delayacct_blkio`, retrieve the aggregated I/O waiting time (`delayacct_blkio`), representing the time the process spent waiting for block I/O operations.

- 5) **Compute Waiting Time in Ticks:** The total waiting time W (in clock ticks) is calculated by subtracting all CPU execution times and I/O wait times from the elapsed time:

$$W = E - (U_t + K_t + CU_t + CK_t + \text{delayacct_blkio})$$

- 6) **Convert Waiting Time to Seconds:** To express the waiting time in seconds, divide W by HZ:

$$T = \frac{W}{\text{HZ}}$$

Where:

- T : Waiting time in seconds.
- HZ: Number of clock ticks per second (system dependent).
- U_t : User mode CPU time consumed by the process (in ticks).
- K_t : Kernel mode CPU time consumed by the process (in ticks).
- CU_t : User mode CPU time consumed by child processes (in ticks).
- CK_t : Kernel mode CPU time consumed by child processes (in ticks).
- `delayacct_blkio`: Aggregated I/O waiting time (in ticks).
- `starttime`: Process start time (in ticks).

This method, utilizing the `proc` filesystem, ensures a precise calculation of the time a process spends waiting in the ready queue by systematically excluding periods spent executing on the CPU (in both user and kernel modes) and waiting for I/O operations. Consequently, the computed waiting time accurately reflects the scheduling delays and resource contention experienced by the process during its lifecycle.

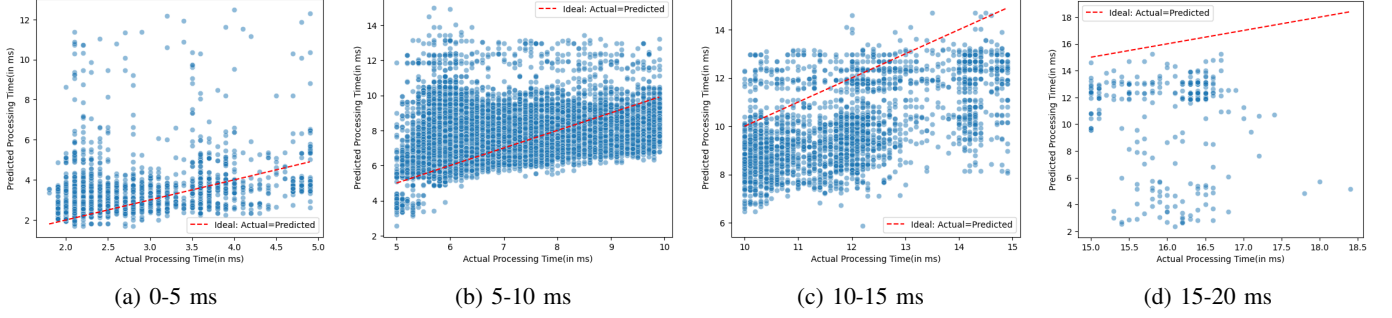


Fig. 1: Regression model performance across different temporal bins (0–5 ms, 5–10 ms, 10–15 ms, and 15–20 ms) for instance segmentation tasks.

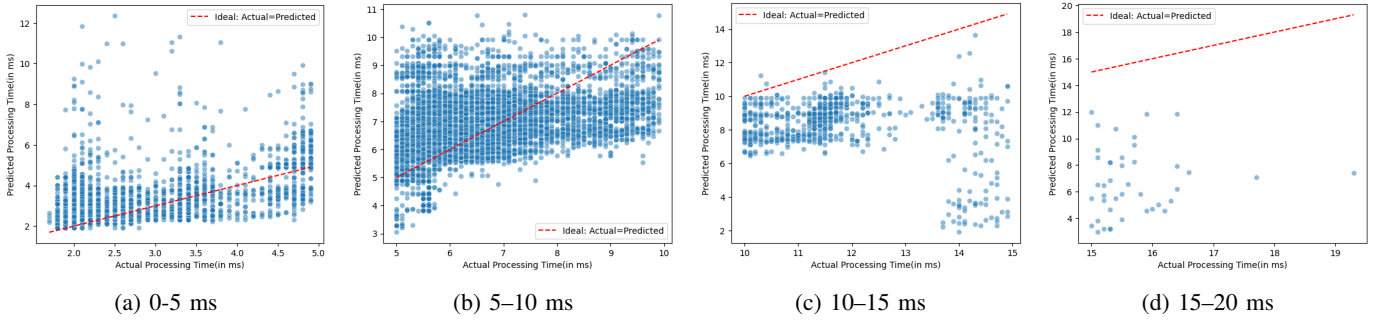


Fig. 2: Regression model performance across different temporal bins (0–5 ms, 5–10 ms, 10–15 ms, and 15–20 ms) for object detection tasks.